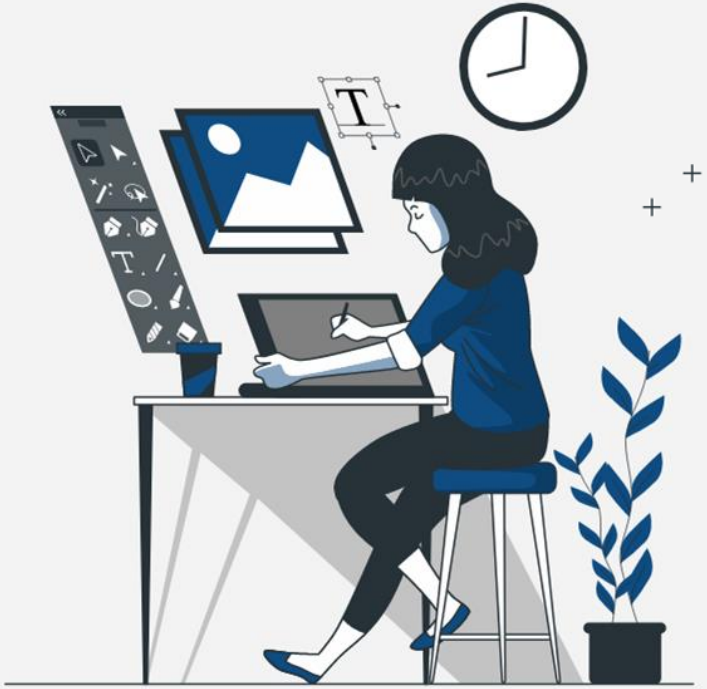
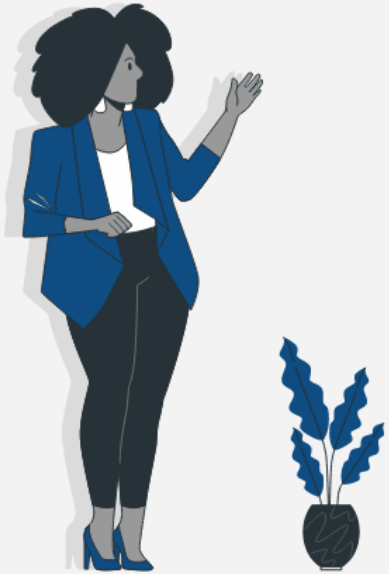
Software Maintenance

Impact Analysis





01

Impact Analysis

Introduction

You can enter a subtitle here if you need it

Introduction & Importance of Impact Analysis

- **Definition**:
  - Impact analysis is the process of assessing the consequences of a software change request (CR) before implementation.
- **Why Impact Analysis Matters**:
  - **Cost Estimation** → Helps estimate the cost of executing a change request (CR).
  - **Critical Impact Detection** → Determines whether critical system components will be affected.
  - **Change History Tracking** → Keeps a record of previous changes for future reference and analysis.
  - **Understanding Structural Impact** → Analyzes how modifications relate to software structure.
  - **Regression Testing Scope** → Identifies which parts of the system require re-testing after a change.

Overview of the Impact Analysis Process

1. Identifying Potentially Impacted Components
2. Analysing Dependencies
3. Predicting Direct & Indirect Impact
4. Evaluating Impact Scope (Risk & Effort Estimation)
5. Decision-Making & Regression Testing.

02

Impact Analysis Process

Starting Impact Set (SIS)
Candidate Impact Set (CIS)
Discovered Impact Set (DIS)
Actual Impact Set (AIS)
False Positive Impact Set (FPIS)

The Impact Analysis Process

1. **Before implementation** - Assess the scope of the change.
   - **SIS (Starting Impact Set)**
     - Identify the initial set of potentially impacted components based on the change request (CR).
2. **During implementation:** Identify ripple effects and update necessary components.
   - **CIS (Candidate Impact Set)**
     - Predict which components are likely to be impacted based on dependencies and traceability analysis.
   - **DIS (Discovered Impact Set)**
     - Identify additional components impacted during the actual change process that were not in CIS.
3. **After implementation:** Verify accuracy using testing & validation.
   - **AIS (Actual Impact Set)**
     - The final set of components actually modified after the change request has been implemented.
   - **FPIS (False Positive Impact Set)**
     - Components wrongly identified as impacted but not actually modified.

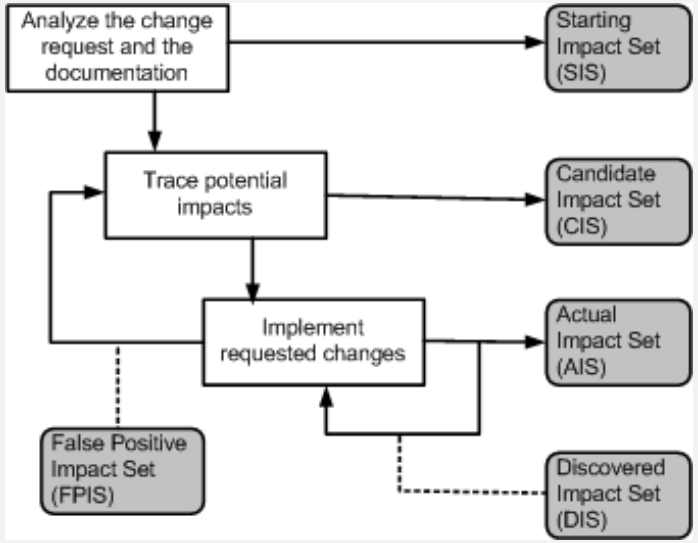


Figure 9.1 Impact analysis process ©IEEE, 2008

Defining SIS & CIS

1. Starting Impact Set (SIS) – Before Implementation

- The initial set of components suspected to be impacted by a CR.
- Derived from code, documentation, and design artifacts.
- Used to define the starting point for analysis.

SIS (Initial identification):

```
# Components directly mentioned in change request
auth_module.login()
auth_module.validate_credentials()
```

CIS (After analysis):

```
# Direct dependencies
auth_module.login()
auth_module.validate_credentials()
session_manager.create_session()

# Indirect dependencies
user_profile.get_login_attempts()
security_logger.log_authentication()
```

2. Candidate Impact Set (CIS) – During Implementation

- The estimated set of impacted components based on analysis techniques.
- Helps prioritize components for further analysis before making actual changes.
- Goal: Reduce false positives and focus on relevant changes.
- **Direct & Indirect Impacts in CIS:**
  - Direct impact: Component A calls Component B → Change in A affects B.
  - Indirect impact: Component B influences Component C through A.

Defining DIS & AIS

3. Discovered Impact Set (DIS) – During Implementation

- The set of components found to be impacted during actual implementation but were not initially identified in CIS.
- Helps refine impact analysis models for future changes.

4. Actual Impact Set (AIS) – After Implementation

- The final set of components that were actually modified as a result of the CR.
- Used to validate predictions from CIS & DIS.

5. False Positive Impact Set (FPIS) – After Implementation

- set of objects (or components) estimated to be impacted by an implementation of a CR but not actually impacted by the CR.
- Used to improve precision of future impact analysis
- Formula for FPIS: **FPIS = (CIS ∪ DIS) \ AIS**
  (where ∪ represents union and \ represents set difference)
- The error in impact estimation can be computed as **(|DIS| + |FPIS|)/|CIS|.**

03

Evaluating Impact Analysis Effectiveness

Recall

Precision

Metrics for Evaluating Impact Analysis

Two traditional information retrieval metrics applied to evaluate the effectiveness of impact analysis, **recall** and **precision**.

1. **Recall** → Measures completeness of identified impacts.
   - It represents the fraction of actual impacts contained in CIS, and it is computed as the ratio of |CIS ∩ AIS| to |AIS|.

$$\text{Recall} = \frac{|CIS \cap AIS|}{|AIS|}$$

   - The value of recall is 1 when DIS is empty.
   - This shows **how well** the approach **captures** all actual impacted components (AIS).
   - A **Higher recall** means the approach successfully finds most of the actual impacted components.

2. **Precision** → Measures accuracy of predicted impacts.
   - It represents the fraction of candidate impacts that are actually impacted, and it is computed as the ratio of |CIS ∩ AIS| to |CIS|.

$$\text{Precision} = \frac{|CIS \cap AIS|}{|CIS|}$$

   - For an empty FPIS set, the value of precision is 1.
   - This shows **how accurate** the **predicted** impacts (CIS) are in relation to actual changes (AIS).
   - A **high precision** means that most of the predicted impacted components (CIS) were indeed actually impacted (AIS).

Example of Precision & Recall

Example Scenario:

- Rohan modifies a legacy system using a heuristic tool for change propagation:
- **Change Set (Actual Changes):** A, B, C, D
- **Predicted Set:** B, X, Y, W, D
- **Occurred Set:** B, C, D (excluding A, as it was initially identified by Rohan and not predicted by the tool)

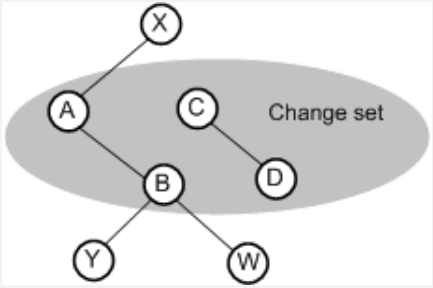


Figure 6.14 Change propagation flow for a simple example ©IEEE, 2004

- **Key Formulas and Calculations:**
- **Precision: Fraction of predicted changes that are correct.**

$$\text{Precision} = \frac{|\text{Predicted Set} \cap \text{Occurred Set}|}{|\text{Predicted Set}|}$$

$$\text{Precision} = \frac{|\{B, X, Y, W, D\} \cap \{B, C, D\}|}{|\{B, X, Y, W, D\}|} = \frac{2}{5} = 0.4$$

- **Recall:** Fraction of actual changes that were predicted by the tool.

$$\text{Recall} = \frac{|\text{Predicted Set} \cap \text{Occurred Set}|}{|\text{Occurred Set}|}$$

$$\text{Recall} = \frac{|\{B, X, Y, W, D\} \cap \{B, C, D\}|}{|\{B, C, D\}|} = \frac{2}{3} = 0.67$$

Insights:

- **Moderate Recall (67%):** Shows decent coverage of actual changes, still missed one-third of necessary changes and it demonstrates reasonable but imperfect change detection
- **Low Precision (40%):** Indicates significant over-prediction, More than half of predicted changes were unnecessary and it shows inefficiency in the impact analysis process
- This indicates that while the tool is moderately effective at identifying required changes, it needs improvement in reducing false positives to be more efficient for maintenance work.

Adequacy & Effectiveness of Impact Analysis Approaches

1. Adequacy

- The ability of the approach to identify all the affected elements to be modified. Ideally, **AIS $\subseteq$ CIS**.
- Adequacy is repressed in terms of a performance metric called **inclusiveness**, as follows:

$$Inclusiveness = \begin{cases} 1 & \text{if AIS} \subseteq \text{CIS} \\ 0 & \text{otherwise} \end{cases}.$$

- The concept of adequacy is essential to assessing the quality of an impact analysis approach.
- An inadequate approach is in fact useless, as it provides the maintenance engineer with incorrect information.
- For example, if inconclusive is 0, the approach cannot be used because it does not provide the maintenance engineer with all the components to be analyzed.

Adequacy & Effectiveness of Impact Analysis Approaches

2. Effectiveness

- The ability of an impact analysis technique to generate results, that actually benefit the maintenance tasks, is known as its effectiveness.
- Effectiveness is expressed in terms of three fine grained characteristics as follows:
  a. Ripple-sensitivity - Ability to track ripple effect.
  b. Sharpness - Minimizing unnecessary impact components.
  c. Adherence - Ensuring impact predictions match actual results.

- **Ripple-sensitivity**
  - Ripple-sensitivity implies producing results that are influenced by ripple effect.

Concept of Ripple Effect

2. Effectiveness

a. Ripple-sensitivity

- The set of objects that are directly affected by the change is denoted by DISO (Directly Impacted Set of Objects).
- Similarly, the set of objects that are indirectly impacted by the change is denoted by IISO (Indirectly Impacted Set of Objects).
- The cardinality of IISO is an indicator of ripple effect.
- The software maintenance personnel expect that the cardinality of IISO is not far from the cardinality of DISO.
- Therefore, the concept of Amplification, as defined below, is used as a measure of Ripple-sensitivity.

$$Amplification = \frac{|\,IISO\,|}{|\,DISO\,|} \longrightarrow 1,$$

where | . | denotes the cardinality operator.

Concept of Sharpness & Adherence

2. Effectiveness

a. Sharpness

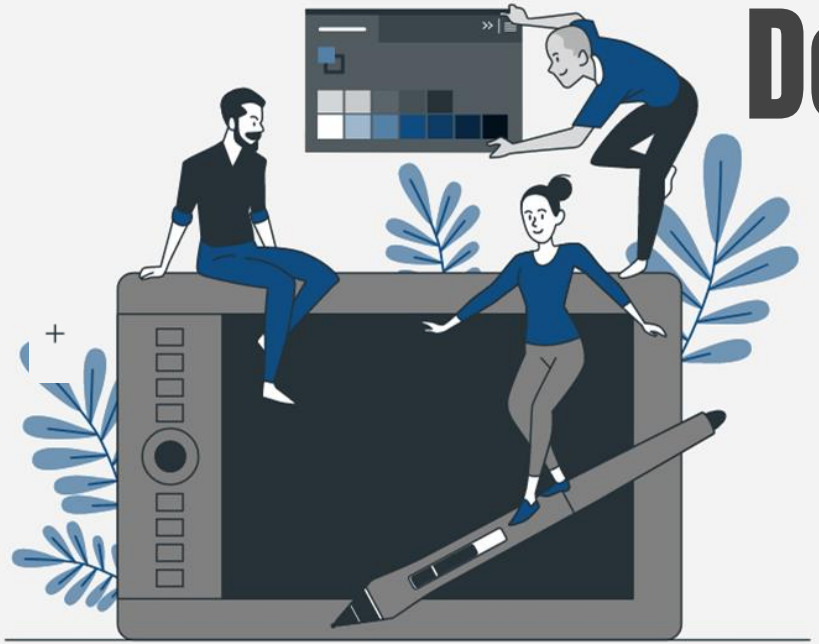
- Sharpness is the ability of an impact analysis approach to avoid having to include objects in the CIS that are not needed to be changed.
- Sharpness is expressed by means of Change Rate as defined below:

$$ChangeRate = \frac{|\ CIS\ |}{|\ System\ |}.$$

- ChangeRate falls in the range from 0 to 1.

a. Adherence

- Adherence is the ability of the approach to produce a CIS which is as close to AIS as possible.
- A small difference between CIS and AIS means that a small number of candidate objects fail to be included in the actual modification set.
- Adherence is expressed by S-Ratio as follows: $S\text{-}Ratio = \frac{|\ AIS\ |}{|\ CIS\ |}.$
- S-Ratio takes on values in the range from 0 to 1.



04

Dependency-Based Impact Analysis

Call Graph Approach
Program Dependency Diagram (PDG)

Dependency-based Impact Analysis

- In general, source code objects are analyzed to obtain vertical traceability information.
- Dependency based impact analysis techniques identify the impact of changes by analyzing syntactic dependencies, because syntactic dependencies are likely to cause semantic dependencies.
- Two traditional impact analysis techniques are explained:
  - based on call graph.
  - based on dependency graph.

Call Graph Approach

- A call graph is a directed graph in which a node represents a function, a component, or a method.
- An edge between two nodes A and B means that A may invoke B
- Programmers use call graphs to understand the potential impacts that a software change may have.
- Let P be a program, G be the call graph obtained from P, and *p* be some procedure in P.
- A key assumption in the call graph-based technique is that some change in *p* has the potential to impact changes in all nodes reachable from *p* in G.
- Limitations:
  - Does not track data dependencies.
  - Cannot detect indirect impact propagation.

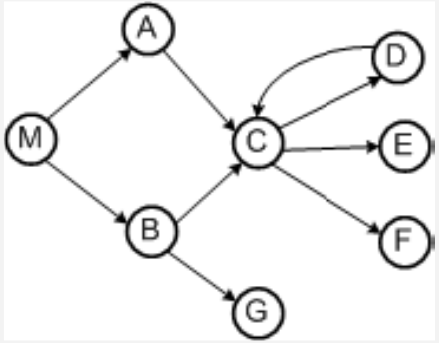
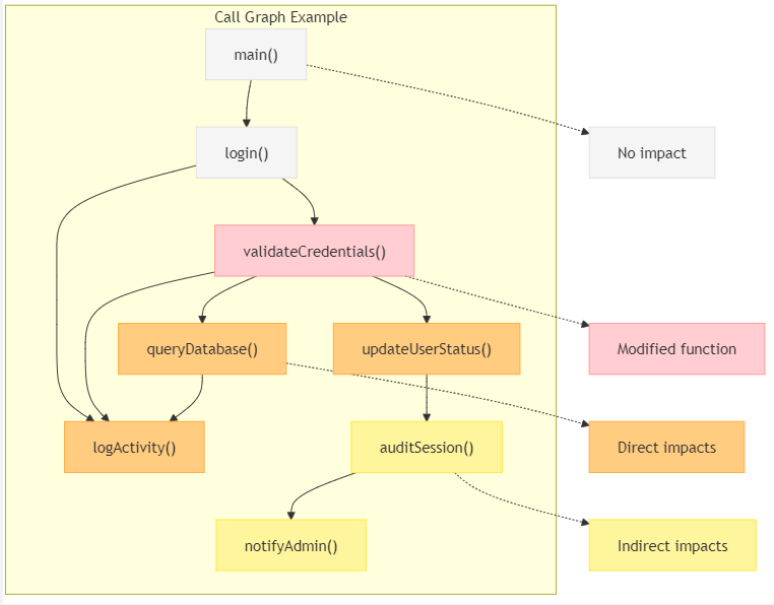


Figure 6.7 Example of a call graph ©IEEE, 2003

Call Graph Approach Example

1. **Structure:**
   - Each node represents a function (like main(), login(), etc.)
   - Arrows show which function calls which other functions
   - The direction of arrows shows the flow of execution
2. **Example Scenario:** Let's say we have a simple login system where:
   - main() calls login()
   - login() calls validateCredentials() and logActivity() .. Etc.
3. **Impact Analysis:** If we modify the validateCredentials() function (red), the changes could potentially impact:
   - Direct Impacts (orange):
     - queryDatabase()
     - logActivity()
     - updateUserStatus()
   - Indirect Impacts (yellow):
     - auditSession()
     - notifyAdmin()
   - No Impact (grey):
     - main()
     - login()



3. **Impact Propagation:**
   - First Level: Direct impacts affect functions immediately called by validateCredentials()
   - Second Level: Indirect impacts show how changes cascade through function chains
   - The chain of impact goes: validateCredentials() → updateUserStatus() → auditSession() → notifyAdmin()
3. **Practical Applications:**
   - Developers can quickly identify which functions need review after modifying validateCredentials()
   - Testing teams can focus on both directly and indirectly impacted functions
   - Helps in estimating the scope of changes and required testing effort
   - Supports better risk assessment when making changes to the codebase

Program Dependency Graph (PDG)

- In the program dependency graph (PDG) of a program:

  (i) each simple statement is represented by a node, also called a vertex;

  (ii) each predicate expression is represented by a node.
- There are two types of edges in a PDG:
  - data dependency edges - When one variable depends on another.
  - control dependency edges - Execution order depends on a condition.
- Let v_i and v_j be two nodes in a PDG.
  - If there is a data dependency edge from node v_i to node v_j , then the computations performed at node v_i are directly dependent upon the results of computations performed at node v_j .
  - A control dependency edge from node v_i to node v_j indicates that node v_i may execute based on the result of evaluation of a condition at v_j .
- Advantage over Call Graph:
  - PDG can track deeper relationships between components.

```
begin
S1:     read(X)
S2:     if (X < 0)
        then
S3:         Y = f1 (X);
S4:         Z = g1 (X);
        else
S5:         if (X = 0)
            then
S6:             Y = f2 (X);
S7:             Z = g2 (X);
            else
S8:             Y = f3 (X);
S9:             Z = g3 (X);
            end_if;
        end_if;
S10:    write(Y);
S11:    write(Z);
end.
```

FIGURE 6.9 Example program. From Reference 31. © 1990 ACM

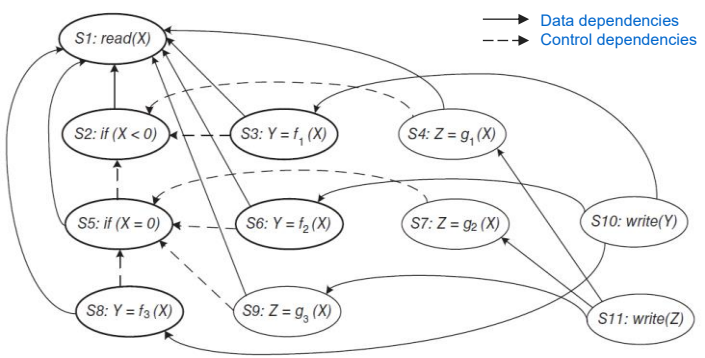


FIGURE 6.10 Program dependency graph of the program in Figure 6.9

Program Dependency Graph (PDG) Example

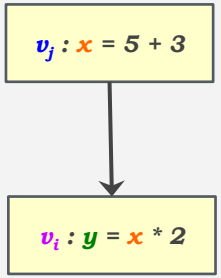
1. **Nodes in the PDG**:
   Think of nodes v_i and v_j as actions or computations in your program. Each node represents a specific operation, like calculating a value or making a decision.

2. **Data Dependency Edge:**
   A data dependency exists when the **result** of a computation in one node (v_j) is needed for the computation in another node (v_i).

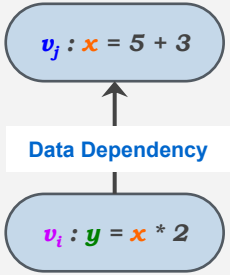
   Example:
   - Node v_j : Compute x = 5 + 3.
   - Node v_i : Compute **y** = x × 2.
   - Here, vi depends on vj because it needs the value of **x** calculated in v_i.

3. **Control Dependency Edge:**
   A control dependency exists when the execution of one node (v_i) depends on a decision made in another node (v_j).
   Example:
   - Node vj : If (condition is true), do something.
   - Node v_i : Executes only if v_j's condition is true.
     Here, v_i will execute based on the result of the condition evaluated in v_j.

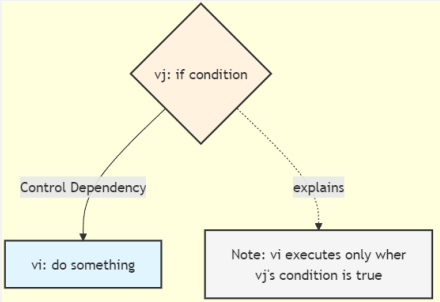
Flow chart



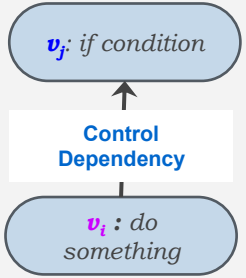
Data Dependency Example



Flow chart



Control Dependency Example



Comparing Call Graph Vs PDG

- **Key Differences:**

Feature	Call Graph	Program Dependency Graph (PDG)
Focus	Function calls & invocation paths	Control & data dependencies
Granularity	Captures only function-level relationships	Both function & variable dependencies
Impact Analysis Use	Identifies directly impacted functions	Identifies both direct & indirect impact propagation
Impact Scope	Only detects function call-based changes	Direct & indirect propagation
Computational Cost	Low (fast & lightweight)	High (complex & expensive)

- **Why is PDG More Effective?**
  - Tracks how variables change, not just function calls.
  - Identifies hidden dependencies in program logic.
  - Captures ripple effects beyond direct function calls

- **When to Use Each?**
  - Use Call Graph → Quick analysis of function-level dependencies.
  - Use PDG → Detailed analysis of data flow & program structure

05

Ripple Effect in Impact Analysis

Concept of Ripple Effect

- **Definition**

  A change in one part of the system propagates to other components, often leading to unintended modifications.

- **Key Characteristics:**
  - Direct Impact: Affected components immediately linked to the change.
  - Indirect Impact: Further dependencies affected through transitive relationships.
  - Affects Both Code & Non-Code Artifacts: Impacts code, tests, documentation, and configurations.

- **Example:**
  - Change: Modify calculateTax().
  - Impact: Affects checkout total, invoice generation, tax reporting, and customer receipts.

- **Risk:** Without proper analysis, ripple effects can cause unexpected failures and high maintenance costs.

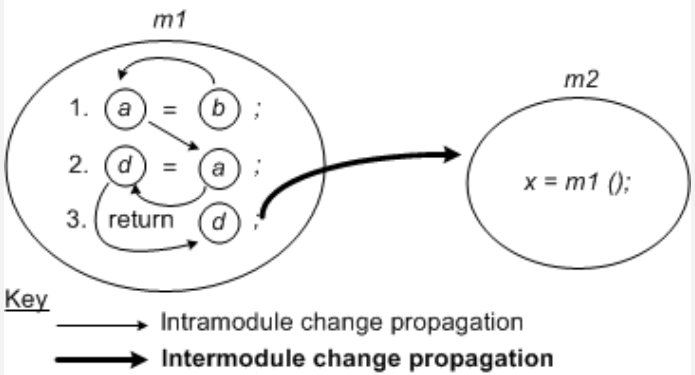


Figure 6.12 Intramodule and intermodule change propagation ©Wiley, 2001

In Figure 6.12, within module m1, 3 lines of code reference variables **a**, **b**, and **d**, illustrating both intra-module and inter-module ripple effects.

- A change in **b** at line (1) affects **a**, which propagates in line (2).
- In line (2), **a** influences **d**, extending the impact to line (3).
- Since these changes occur within module **m1**, they represent an intra-module ripple effect.
- If d is accessed by another module, the effect extends beyond **m1**, causing an inter-module ripple effect where modifications impact external components.

This demonstrates how local changes can escalate, affecting both internal and

Measuring and Analyzing Ripple Effect

- ## How to Measure Ripple Effect?
  - **Direct Analysis**: Function calls, data flow, shared resource dependencies.
  - **Indirect Analysis**: Interface changes, secondary dependencies, cross-module interactions.
- ## Key Metrics:
  - **Change Scope** → Number of affected components.
  - **Depth of Propagation** → Levels of impact.
  - **Complexity of Affected Components** → Higher complexity = higher risk.
- ## Amplification Factor :

$$Amplification = \frac{|\ IISO\ |}{|\ DISO\ |}$$

IISO = *Ind*irectly Impacted Set of Objects (components)

DISO = Directly Impacted Set of Objects (components)

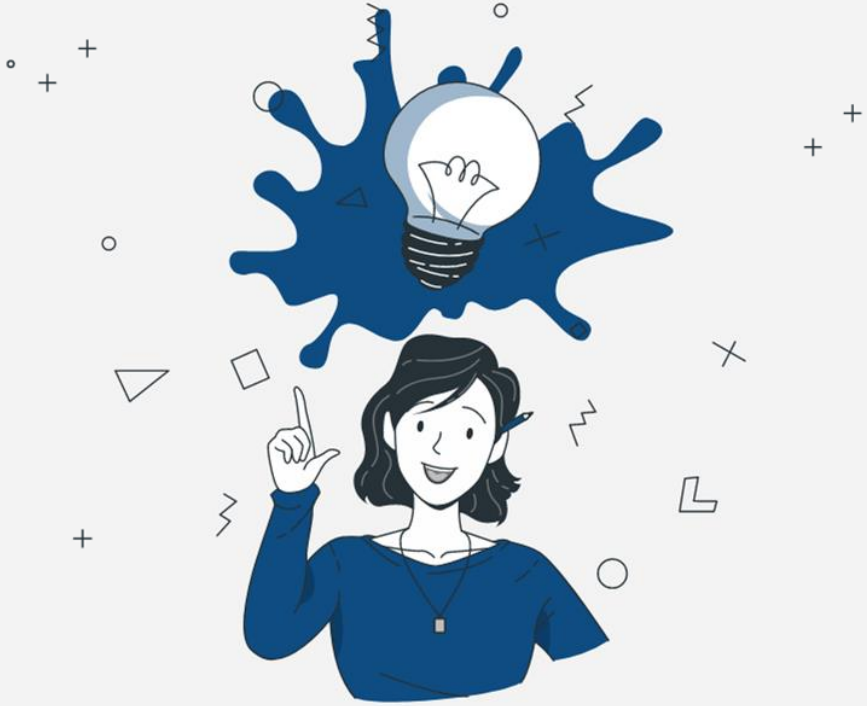
  - **Higher value** → Greater risk of cascading failures.
- ## Methods for Analysis:
  - Static Analysis: Examines code structure.
  - Dynamic Analysis: Tracks runtime dependencies.
  - Historical Analysis: Uses past changes to predict impact.

Managing and Minimizing Ripple Effect

Strategies to Reduce Ripple Effect

- **Architectural-Level**:
  - Modular Design → Reduce interdependencies.
  - High Cohesion, Low Coupling → Keep components independent.
- **Implementation-Level:**
  - Encapsulation & Dependency Injection → Minimize direct dependencies.
  - Clear Interface Contracts → Prevent unintended changes.
- **Best Practices:**
  - Perform Impact Analysis Before Changes.
  - Document Dependencies → Maintain traceability.
  - Automate Regression Testing → Catch unintended side effects.
- **Recommended Tools:**
  - Dependency Analysis: SonarQube, JArchitect.
  - Version Control & Change Tracking: Git history, CI/CD pipelines.
- Proper impact analysis & testing prevent unintended failures, ensuring system stability.

06



Change Propagation Models & Heuristics

What is Change Propagation?

Definition

- **Change propagation** refers to how a modification in one software component triggers cascading changes in other related components within the system.
- This phenomenon is a critical aspect of impact analysis, as it helps determine the extent to which a change will affect the system.

Why is Change Propagation Important?

- Predicting Ripple Effects: Understanding how changes will propagate helps predict unintended impacts across the software system.
- Scope of Impact: By analyzing propagation, teams can estimate which components need testing and how deeply the changes will affect the system.
- Efficiency in Maintenance: Reduces the time and cost of manual testing by ensuring all affected areas are identified.

Change Propagation Process

- The process involves the following steps:
  1. Identify the source of the change (e.g., a function, a module, a class).
  2. Analyze direct dependencies (components that directly use or call the source component).
  3. Trace indirect dependencies (components affected indirectly through chains of relationships).
  4. Map out the final impact set, including all affected components that need modification or retesting.

Change Propagation Model

Figure 9.13 illustrates the structured flow of change propagation through a software system. The model highlights the stages through which a change passes, helping to map its impact across the system.

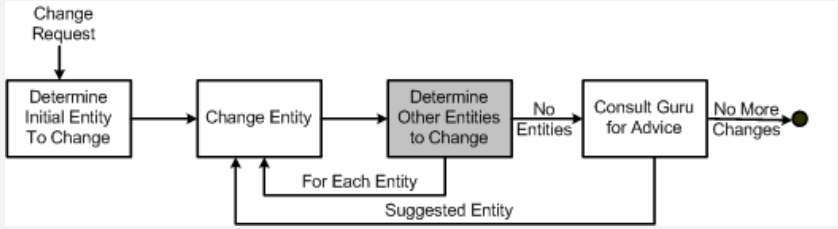


Figure 9.13 Change propagation model ©IEEE, 2004

- Key Elements in the Change Propagation Model:
  1. Determine Initial Entity to Change:
     - Identify the starting point of the change (the source component) based on the Change Request (CR).
     - This could be a function, method, or module that needs to be modified.
  2. Change Entity:
     - Modify the identified source component according to the change request.
     - This step involves updating the code or system to reflect the change.
  3. Determine Other Entities to Change:
     - Analyze direct and indirect dependencies using static and dynamic analysis techniques.
     - Identify related components that are impacted by the change, whether directly or indirectly.
  4. Consult Guru for Advice:
     - When uncertainty arises, consult with domain experts (referred to as "gurus") to ensure no components are overlooked.
     - Gurus provide insights into historical changes or hidden dependencies that may not be obvious in the initial analysis.

Heuristics for Change Propagation

What Are Heuristics?

- Rules of thumb to predict impacted components based on past experience or dependencies.
- Helps prioritize components for modification.

Types of Heuristics

- Historical: Based on past changes (components modified together).
  E.g. If Module A and Module B were modified together, they likely will be again.
- Dependency-Based: Based on function calls and data flow.
  E.g. A change in Function A may require changes in Function B.
- Textual: Uses name similarity or logical connection.
  E.g. Modifying updateInvoice() may also require changes to generateInvoice().

Why Heuristics Matter

- Improves Efficiency - Focuses on likely impacted components.
- Enhances Accuracy - Prioritizes critical areas for modification.

Heuristic Information Sources & Pruning Techniques

Information Sources for Heuristics

- Heuristic-based approaches rely on multiple information sources to predict changes in the system:
  1. Entity Info: Based on co-changes (components changed together).
     E.g. Module A and Module B frequently change together.
  2. Developer Info: Based on the developer's previous work.
     E.g. If a developer works on Module X, future changes may also affect it.
  3. Process Info: Based on change patterns due to business needs.
     E.g. Payment system changes often affect accounting.
  4. Textual Info: Uses name similarities between components.
     E.g. Changes to updateCustomer() may require validateCustomer() to be updated.

Pruning Techniques

- Pruning techniques focuses on components that have been modified together frequently in the past.
- Essential in impact analysis, it helps narrow down the set of components that need to be analyzed.
  1. Frequency-Based Pruning - Focuses on components frequently modified together.
  2. Recency-Based Pruning - Prioritizes recently changed components, assuming that recently changed components are more likely to be impacted again.
  3. Random Selection Pruning - Selects components randomly when no data is available.

07

Best Practices & Industry Applications

Best Practices for Effective Impact Analysis

Key Best Practices in Impact Analysis

1. **Comprehensive Impact Analysis Before Changes**
   - Always conduct a thorough impact analysis before implementing any change to understand how it will affect the system.
2. **Document Dependencies**
   - Maintain clear documentation of all dependencies (internal and external) between components, including data, functions, and modules.
   - Use tools like dependency graphs to visualize relationships.
3. **Automated Regression Testing**
   - Implement automated regression tests to quickly identify unintended consequences of changes, ensuring that no broken functionality exists after updates.
4. **Continuous Integration (CI) and Continuous Deployment (CD)**
   - Integrate impact analysis and testing into CI/CD pipelines to catch issues early and continuously monitor the impact of changes across the system.
5. **Use Traceability Tools**
   - Leverage traceability tools to track relationships between requirements, code, tests, and design, ensuring that all impacted areas are reviewed and updated.

Tools Used in Industry

- These tools provide insights into dependencies and help automate testing and version tracking, ensuring early detection of issues and efficiency in maintaining software

- **Dependency Analysis Tools**
  - **SonarQube**: Provides static analysis and highlights dependency issues and potential risks.
  - **JArchitect**: Analyzes Java project dependencies to detect architectural violations and potential ripple effects.
  - **Lattix**: Focuses on modular dependency analysis to track impact across multiple modules.

- **Version Control & Change Tracking**
  - **Git**: Provides version history and helps track dependencies between components based on commit data.
  - **JIRA**: Tracks change requests and links them with test cases, requirements, and code changes to ensure all impacted components are reviewed.

- **Automated Testing Tools**
  - **Selenium**: For automated UI testing to detect ripple effects in front-end changes.
  - **JUnit**: For automated unit testing, ensuring that core functionality is unaffected by changes.
  - **TestNG**: Used for running integration tests that check for cross-module impacts.

Common Challenges in Real-World Impact Analysis

Key Challenges in Impact Analysis

1. Legacy Systems
   - Understanding dependencies in legacy systems can be difficult due to lack of documentation and obsolete technology.
   - Impact analysis can become a challenge when system architecture is not well-documented.
2. Poor Documentation
   - Inadequate or outdated documentation makes it harder to track dependencies accurately, leading to potential oversight of impacted components.
3. High System Complexity
   - In large and complex systems, determining all affected components requires more resources and time.
   - Hidden dependencies often create unforeseen ripple effects.
4. Inaccurate Dependency Mapping
   - Outdated or incorrect dependency maps can lead to misidentification of impacted components. Without proper tools and continuous updates, dependencies can be misleading.
5. Resistance to Change
   - Human factors such as resistance to change and lack of collaboration across teams can slow down the implementation of impact analysis processes.

Overcoming These Challenges

- Modern tools like SonarQube, JIRA, and Git provide solutions for legacy systems and improve documentation accuracy.
- Collaboration and clear communication among development, testing, and operations teams help ensure successful change implementation.

THANK YOU

Do you have any questions?

Acknowledgement:

Priyadarshi Tripathy, & Kshirasagar Naik. (2015). Software evolution and maintenance : a practitioner’s approach. Wiley

Chefranov, A. (2024). Lecture 4 - Impact Analysis. Eastern Mediterranean University.